KIUWAN REPORT

# VULNERABILITIES REPORT

Application: jVtiELjw1AoJoN4n0v10S9Zd0xVHMi8/5gLA/3TTv2M=
Analysis Label: Defensoria-Quejoso
Analysis Date: 2026/05/06 09:38
Report Date: 2026/05/06 01:39

# APPLICATION & ANALYSIS INFORMATION

This software has been scanned and vulnerabilities checked for based on the following standards: OWASP, CWE, SANS 25, PCI-DSS, HIPAA, WASC, MISRA-C, BIZEC, ISO 25000, ISO 9126, CERT-C, CERT-J. Kiuwan has analyzed for you a set of source files. In order to put this analysis and your results in context, here are some statistics:

Notes:

- unparsed files are the source code files of your application that kiuwan engine couldn't read during the analysis process.
- unrecognized files are the ones that were in your analyzed path but they aren't source code or they contains code in languages that Kiuwan doesn't support yet.

## Analysis Info

| | |
|---|---|
| label | Defensoria-Quejoso |
| date | 2026/05/06 01:30 |
| ordered by | Jose Bryan Omar Jimenez Velazquez |
| encoding | UTF-8 |
| analyzed path | - |
| languages found | java |
| analyzed files | 43 |
| unparsed files | 0 |
| unrecognized files | - |

## Application Info

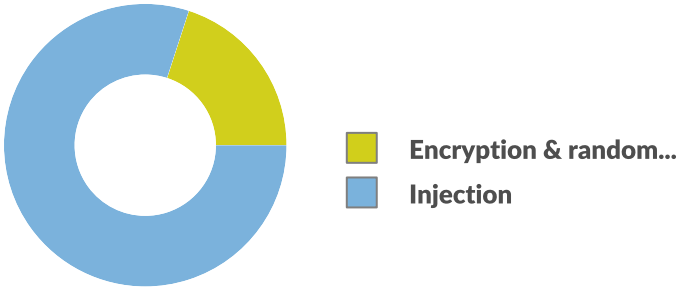
| | |
|---|---|
| name | Quejoso_Defensoria |
| Bussiness value | critical |
| Times analyzed | 1 |

## Model Info

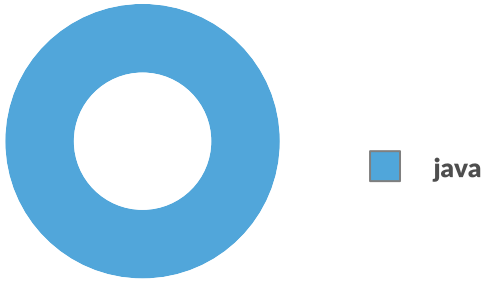
| | |
|---|---|
| model name | OWASP-benchmark |
| model version | 3 |
| engine version | master.p708.q13635.a1914.i675 |
| active rules | 11 |
| active metrics | 62 |

# VULNERABILITIES DISTRIBUTION

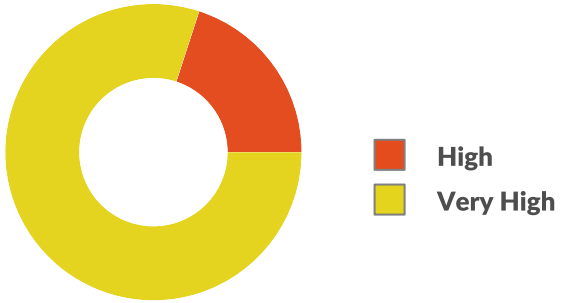
## BY TYPE




## BY LANGUAGE




## BY PRIORITY




## VIOLATED RULES

2

## SECURITY RATING

## VULNERABILITIES

5

## VERY HIGH

4

# VULNERABILITIES

| FILES | #VULNERABILITY | RULE | CHARACTERISTIC |
|---|---|---|---|
| 4 | 4 | Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') | Security |

| PRIORITY | CWE | VULNERABILITY TYPE | LANGUAGE |
|---|---|---|---|
| | 79 | Injection | java |

Software places user-controlled input in page content. An attacker could inject browser script code that is executed in the client browser. The end-user is the attacked subject, and the software is the vehicle for the attack.
There are two main kinds of XSS:

* Reflected XSS: Attacker causes victim to supply malicious content to a vulnerable web application, which renders HTML content embedding a malicious script executed in victim's browser. A variation of this is named DOM-based XSS, where the vulnerable software does not generate content depending on user input but include script code that use user-controlled input.

* Persisted XSS: Attacker provides malicious content to vulnerable application. When other user access to vulnerable pages that embed without proper neutralization the attacker content, script code is executed in the victim's browser.

The script executed in the victim's browser could perform malicious activities.

Many browsers could limit the damage via security restrictions (e.g. 'same origin policy'), but user browsers generally allow scripting languages (e.g. JavaScript) in their browsers (disabling JavaScript severely limits a web site).

1 Sink in file src/main/java/ipn/escom/defensoria/quejoso/controller/AuthController.java

48 klzA4oXDqXHf4SQekYrqFxXEgcCUyF1g/L+GOpWghjziy3mdjX1glbdycly+pTK6scyHbXr0V6OgV6BMrOODzg==

Propagation path #1

Source src/main/java/ipn/escom/defensoria/quejoso/service/UsuarioService.java

114 lUpGnuKf/jxW9juam7h1vxqrTXgmWDviFsLiJVhLt/GAMyJIfGMe2KXcEQBBdwWBffpEJi/KbENjZVmblWKDLw==

---

1 Sink in file src/main/java/ipn/escom/defensoria/quejoso/controller/PerfilController.java

24 klzA4oXDqXHf4SQekYrqF67zzFydLFQSS6RL+N9zcjJ/AODXCwGiShh/SW/TjYRggi4zVq7U7gRwyZjol/5tcSoQVUmjQ9pEa9GY8hIYspWyjk3SQJZe8pR6eeGXNxqb

Propagation path #1

Source src/main/java/ipn/escom/defensoria/quejoso/service/UsuarioService.java

114 lUpGnuKf/jxW9juam7h1vxqrTXgmWDviFsLiJVhLt/GAMyJIfGMe2KXcEQBBdwWBffpEJi/KbENjZVmblWKDLw==

---

1 Sink in file src/main/java/ipn/escom/defensoria/quejoso/controller/QuejaController.java

54 fUdFrQdpKA10cy4zkZwl5CaulCcGB7V2uKjhUXM/rN2ysAHdBeg9Fmi+K2Zdt8zYQp7zdUvRfD4SS7WY3GoVoQ==

Propagation path #1

Source src/main/java/ipn/escom/defensoria/quejoso/controller/QuejaController.java

33 p33G+xCCGafniBrsFXm5Pk/xjMNHOY2VZFC4LO33/QjLU/xyUliako58hB1+WS6i1SNTGjSxx8OWOd2RvD04/A==

---

1 Sink in file src/main/java/ipn/escom/defensoria/quejoso/controller/TramitesController.java

38 klzA4oXDqXHf4SQekYrqF7PKysyfrrk0gzJR+vV3uazFGd5zN8tmxTeO4xZAcSrggu4FSKbtTfCbCnUuLkWn6+O7Mqe7H1B+GSpQlA2uByPqCiF1afurtdfMYlafi5viWnlQS7HJ9PkVOODW+A9C4w==

Propagation path #1

Source src/main/java/ipn/escom/defensoria/quejoso/controller/TramitesController.java

34 rVLpLsN4Tl4gLVYfCItI9d+WfP86rdsRKQhQRLBgd5I=

| FILES | #VULNERABILITY | RULE | CHARACTERISTIC |
|---|---|---|---|
| 1 | 1 | Standard pseudo-random number generators cannot withstand cryptographic attacks | Security |

| PRIORITY | CWE | VULNERABILITY TYPE | LANGUAGE |
|---|---|---|---|
| | 330 | Encryption & randomness | java |

Insecure randomness errors occur when a function that can produce predictable values is used as a source of randomness in a security-sensitive context: security tokens (like anti-CSRF or password-reset tokens), values used in cryptographic operations (session key material, initialization vector in block or stream ciphers), or password seeds.

Computers are deterministic machines, and as such are unable to produce true randomness. Pseudo-Random Number Generators (PRNGs) approximate randomness algorithmically, starting with a seed from which subsequent values are calculated.

There are two types of PRNGs: statistical and cryptographic. Statistical PRNGs provide useful statistical properties, but their output is highly predictable and forms an easy to reproduce numeric stream that is unsuitable for use in cases where security depends on generated values being unpredictable.

Cryptographic PRNGs address this problem by generating output that is more difficult to predict. For a value to be cryptographically secure, it must be impossible or highly improbable for an attacker to distinguish between it and a truly random value.

In general, if a PRNG algorithm is not advertised as being cryptographically secure, then it is probably a statistical PRNG and should not be used in security-sensitive contexts, where its use can lead to serious vulnerabilities such as easy-to-guess temporary passwords, predictable cryptographic keys, or session hijacking, among others.

The rule also checks that the standard java.security.SecureRandom is initialized properly, which basically means that its setSeed() method is not called with any seed generated by custom code, except SecureRandom.getSeed(int) or SecureRandom.generateSeed(int), which are accepted as a proper seed with "adequate entropy".

---

1 Defects in file src/main/java/ipn/escom/defensoria/quejoso/service/UsuarioService.java

170 | 2MbYb1p7heTLOqEfx7uQyr+/qoUJKqfQ+RHMEo3gx2QRBJ76zBHSHSgMXSifslPv7hBwgGO+C/PkquSFA2oJXBXm3EfzT13z5y7p/9/yS43NeMVaSk8AqXqdeYElAGh7


# THANK YOU!

contact@kiuwan.com | Partnership:partners@kiuwan.com | +1 6176073353



kiuwan